

What Does Content Access Buy? Pricing the Observability Ladder for Behavioral Drift Detection

Version 0.4 — working draft (2026-09-21): all five experiment families measured; abstract, cost side and references filled; figures added and Section 4 ordered up the ladder; Section 5 added — the dividing line (executional vs. semantic drift) that sorts every measured number.

Authors: Jürgen Eckel, Joerg Radehaus (KYDE).

Companion to *Behavioral Drift in Autonomous LLM-driven Systems* (../paper/), whose Section 7 numbers are the Lo baselines throughout.

Abstract

Boundary call records — tool name, hashed parameters, status, never content — are the cheapest observation an agent deployment can keep, and our companion survey measured what they detect. This paper prices what each further rung of the observability ladder buys: plain call parameters (L1), tool outputs (L2), reasoning text (L3), on the same corpora, the same splits, and under the same rule that every detector stays deterministic. Measured against the published Lo baselines, the rungs do not buy detection by themselves. Reward hacking stays at 0.0% under L1 token frequencies at every window size; what changes the number is detector *type* — frozen mechanism rules over the same command lines reach 36–73%, and justification patterns over reasoning text 97–100%, a ceiling that holds only for openly narrating agents and collapses to base rate (1.5–1.9%) on a second corpus whose hackers had no reason to narrate. Where the *cause* of drift itself crosses the boundary as content — prompt injection arriving in a tool result — the rung that stores it detects it almost entirely (98.7% of successful-injection runs at 4.5% FPR, against 0–8% at Lo). Most of the human-annotated error mass, however, is semantic judgment that no deterministic detector sees at any rung (content rules at the noise floor on TRAIL; 26.3% detection at 21.7% FPR on the one attribution corpus with a clean side), and which rung sees anything at all is decided by where the scaffold routes its observations. A final experiment prices realism itself, on 5,851 real user sessions with real timelines: the composed-stream protocol of all earlier measurements costs approximately nothing (false-alarm rates match within a few points), per-user behavioral fingerprints are real (within-user JSD 0.15 vs 0.36 between users), but in-control churn on real deployments sits six times above the version-drift detection line measured on benchmark batches — real CLI updates are behaviorally sub-noise, and frozen baselines age within weeks. Across the five families the measured numbers sort along one line, and it is not the rung line: deterministic detection buys real rates exactly where the drift leaves an executional trace in stored bytes and the rules are authored for the deployment; where the failure is a semantic judgment it sees nothing at any rung, and on real timelines in-control churn buries what remains (Section 5). The cost side is measured throughout: 3–45× storage over Lo, deployment-specific detector content that does not transfer, a benign base rate to monitor wherever the detector reads what outsiders write, and the loss of shareability that begins one rung above the boundary.

1. Question and setup

The companion survey measured what boundary call records — tool name, hashed parameters, status, attribution, never content — can and cannot detect, on six public corpora: version drift detects (80–100% once $\text{JSD} \geq 0.02$), in-session injection is near-invisible (0–8% at 2–11% FPR), reward hacking is

invisible (0.0% on two labeled corpora), and the runtime status signal carries ~6% of human-annotated error mass. Those limits were measured, not argued.

This paper asks the follow-up question: **how much detection does each additional observability layer buy, and at what cost?** The layers form a ladder:

RUNG	OBSERVATION	COST ADDED
L0	boundary call records (the companion's object)	none — reference
L1	+ plain-text call parameters	content storage; redaction burden
L2	+ tool outputs / observations	large storage; sensitive content
L3	+ reasoning text (agent messages)	model/provider access; heaviest content

Two rules carry over unchanged from the companion and are the point of the design. First, **same ground truth, same splits, same calibration regime** as the L0 experiments — every number here is directly comparable to a published L0 baseline. Second, **every rung's detectors stay deterministic** (pattern, token, and structure features over stored content; no LLM judgment), or the evidentiary property both papers argue for is spent at the second rung.

2. Experiments

- **E1 — Reward hacking, L1/L3** (Terminal Wrench, TRACE; L0 baseline 0.0%). L1: the same per-run surprisal experiment with the tool vocabulary widened from the leading command token to canonicalized full command tokens. L3: deterministic justification patterns over reasoning text. The source of Terminal Wrench reports explicit justification in the reasoning for ~72% of exploit attempts — one rung above where L0 measured 0.
- **E2 — In-session injection, L2** (AgentDojo; L0 baseline 0–8%). The companion's Section 2.3 observes that the *cause* crosses the boundary: injected content arrives in a tool result. Deterministic content features on inbound tool results, scored against the same executed-injection labels.
- **E3 — Context decay, L1/L2** (TRAIL; L0 bound: status covers ~6%). Token counts as true context length instead of step count; how much of the unannotated ~94% becomes visible with tool outputs in view.
- **E4 — Multi-agent attribution, L3** (Who&When; E4b adds AgentHallu, the first attribution corpus with a clean side, used for paper measurements only — its repository LICENSE says CC BY 4.0 while its project page says CC BY-NC-SA). Sequence plus content on the delegation seam; AgentHallu turns attribution into detection rates.
- **E5 — Real timelines** (SWE-chat: 5,851 real working sessions of 190 users, real wall-clock timestamps; ODC-BY). Null streams, real in-deployment CLI version drift, and fingerprint stability on real multi-session timelines — removing the companion's composed-timeline caveat by measuring it. (TraceLab remains registered but undownloaded: license unclear.)

3. Method notes

- **Content sidecars, not schema changes.** The L0 ledgers of the companion stay untouched and citable. Each rung adds a fingerprinted sidecar file keyed by (run_id, step_index); adapters emit them deterministically from the same sources.
- **Canonicalization is part of the detector**, documented and frozen like a threshold: e.g. L1 command tokens with numeric and hex literals classed as <num>/<hex>, everything else verbatim.
- **The result table** is the companion’s Table 2 rotated: drift type × rung × measured detection at matched FPR.
- **Real timelines change the unit of analysis, not the protocol.** In E5 a deployment is one identified user’s Claude Code use; sessions are the runs, ordered by wall clock instead of by seed. Unattributed sessions (34% of the claude-code ledger’s sessions, carrying 39% of its records) are excluded — merging unknown users into one pseudo-timeline would manufacture drift — and an empty CLI-version string is treated as *unknown*, never as a version value: it can neither vouch for a no-change null block nor count as a verified change. Everything else — 50/25/25 admission/calibration/observation splits, thresholds as max in-control score × margin, both channels — is byte-identical to the companion’s protocol, which is what makes the real-vs-composed comparison a measurement of the caveat rather than a new experiment.

4. Results

(filled as experiments land; each result file lives in ../experiments/results/ and is regenerated by a *driftdetect eval* module)

EXPERIMENT	RUNG	CORPUS	L0 BASELINE	MEASURED	RESULT FILE
E1	L1	Terminal Wrench	0.0% detection	0.0% detection at 0.0% FPR (per-run, 3 models, 10 trials)	l1-terminalwrench-rewardhack.md
E1	L1 sweep	Terminal Wrench	—	FP 100% → 0% as window 50 → 3200; detection 0% at every window	l1-terminalwrench-window-sweep.md
E1b	L1 rules	Terminal Wrench	0.0% (distributional)	36.2–72.9% detection at 0.5–1.6% FPR (held-out split, 3 models)	l1b-terminalwrench-artifacts.md
E1	L3	Terminal Wrench	0.0% detection	97.5–99.7% detection at 0.2–1.2% FPR (held-out split, 3 models)	l3-terminalwrench-rewardhack.md
E1 transfer	L1b + L3	TRACE	0.0% (L0)	L1b 1.9% at 0.4% FPR, L3 1.5% at 1.2% FPR — both collapse	trace-transfer.md
E2	L2	AgentDojo	0–8% detection at 2–11% FPR	98.7% (successful-injection runs) / 86.2% (all attacked) at 4.5% FPR (held-out, 29 pipelines)	l2-agentdojo-injection.md

EXPERIMENT	RUNG	CORPUS	L0 BASELINE	MEASURED	RESULT FILE
E3	L2/L3	TRAIL	status covers ~6% of annotated mass	own-location content rules at the noise floor (GAIA 6.0% vs 5.9% neg; SWE 16.9% vs 13.3%)	trail-content-coverage.md
E3	L1 replay	TRAIL	—	GAIA 31.6% of annotated mass vs 13.4% noise (delta attribution); SWE 11.9% vs 10.0% — nothing	trail-content-coverage.md
E3	L1 tokens	TRAIL	step-index proxy	annotated density per model call FALLS with true context length (GAIA 0.33 → 0.10, SWE 0.43 → 0.28)	trail-content-coverage.md
E4	L3	Who&When (Algorithm-Generated)	seam observability only, no rates	mistake step@1 30.9% (before_first_error) vs 14.5% best positional / 12.0% random; agent@1 55.5%	whowhen-content-attribution.md
E4	L3	Who&When (Hand-Crafted)	—	content predictors at/below baselines; the only lift is structural (first_worker: step@1 18.2% vs 4.8% random, agent@1 59.1%)	whowhen-content-attribution.md
E4b	L1–L3	AgentHallu	first clean side in the family	detection 26.3% at 21.7% FPR held-out (negative margin on 3 of 7 frameworks) — no deterministic separation; E4's causer heuristic collapses	agenthallu-hallucination.md
E5	L0, real order	SWE-chat	companion FPR measured on composed streams only	real ≈ composed: FP 6% / 34% (div/cusum) chronological vs 5% / 30% shuffled at margin 2.0 (65 users) — the caveat prices at ~zero	swechat-real-timelines.md
E5	L0, real version drift	SWE-chat	SWE-bench batches: 80–100% where JSD ≥ 0.02	19 in-deployment CLI updates: detection ≈ no-change control (26% / 47% vs 17% / 55% at margin 2.0); boundary JSD 0.132 vs churn JSD 0.127	swechat-real-timelines.md

EXPERIMENT	RUNG	CORPUS	L0 BASELINE	MEASURED	RESULT FILE
E5	L0, fingerprint	SWE-chat	per-deployment baseline premise (argued)	within-user JSD 0.149 « between- user 0.358 (400- record blocks); frozen baseline ages 0.137 → ~0.2 JSD within ~2 weeks	swechat-real- timelines.md

E1/L1 (2026-09-18). The corpus-scale effect size grows by 3–4× — clean-vs- hacked JSD 0.33–0.38 over 51k–109k-token L1 vocabularies, against 0.09–0.12 over the ~20-tool L0 vocabulary — but per-run surprisal still detects nothing: hacked runs spend their tokens on the same commands, paths, and flags as clean runs, and the shift stays a mixture-level property. The stream experiment adds a negative result about the *detector*, not the corpus: a 50-token window over a vocabulary this size saturates the windowed JSD near its bound, so the divergence channel false-alarms on 100% of null streams (CUSUM: 20–30%) and the calibration-cap logic cannot save it. Widening the observation without rescaling the window buys corpus-scale signal and breaks the run/window-scale machinery — the first measured price tag on the ladder.

E1/L1 window sweep (2026-09-18). Sweeping the divergence window over 50/200/800/3200 tokens (same corpus, splits, calibration) separates the two halves of that breakdown. The calibrated threshold never de-saturates — it sits at the 0.9 cap in 120 of 120 cells, because the in-control maximum times margin 2.0 always exceeds the bound. What changes is the null stream: its windowed JSD falls below the cap as the window grows, so false alarms drop from 100% (window 50–200) through 50–90% (800) to 0–10% (3200). Detection, however, stays at 0% at *every* window — the lone 10% cell (opus, window 800) is one trial alarming 64k tokens after onset, noise at the horizon scale. Reading: window rescaling repairs the false-alarm pathology but buys no detection at L1. The 0.33–0.38 corpus-level JSD never concentrates in any 50–3200-token stretch, because hacked runs spend their windows on the same commands as clean ones; the shift only exists across thousands of runs. The window is not the bottleneck — the observation is.

E1b (2026-09-18). The complement to the sweep: hold the observation at L1 and change the *detector type*. Five frozen mechanism rules over the plain command lines — verifier reconnaissance (find/grep for verify/score/grade artifacts), package-database tampering, version-sniffing fake binaries, fabricated version banners, forced installs — the deterministic analogue of an operator-declared sensitive-artifact list; rules developed on a frozen ~20% dev split (`sha256(run_id) % 5`) and measured held-out only. Held-out: **72.9% / 36.2% / 70.0% detection at 1.6% / 1.2% / 0.5% FPR** (opus / gemini / gpt), dev and eval agreeing. So L1 was never information-free: the same command lines whose token *frequencies* detect 0.0% carry mechanism-level evidence that rules recover — the rung’s value depends on the detector type, not just on the rung. Three qualifications carry the number. Coverage is dominated by a single behavior, verifier recon (69/33/69% of hacked runs; the tampering rules are near-zero-FP but reach only 1–6%), and recon is *pre-exploit* behavior an instructed hacker exhibits freely — a tacit one may not search at all, which the gemini column already shows at 36%. The split separates runs, not tasks, so this is within-task-distribution generalization. And unlike the distributional channels, the rule set is content-bearing and deployment-specific — it must be authored and maintained per scaffold, which is exactly the cost Table 2’s “per-agent task model” line predicted.

E1/L3 (2026-09-18). The top rung on the same corpus: thirteen frozen justification patterns over the reasoning-text sidecar, flag at ≥ 2 distinct matches per run; pattern set and threshold developed on the same frozen dev split as E1b and measured held-out: **97.5–99.7% detection at 0.2–1.2% FPR** across the three models, with dev and eval rows agreeing (no overfitting signature). Where L0 and L1 both measured 0.0%, the reasoning channel detects nearly everything — because Terminal Wrench’s hacked runs are goal-conditioned to exploit and narrate it openly (“bypass”, “fake”, “verifier weakness”). The honest reading is a *ceiling*: this is what reasoning access buys against an agent with no incentive to hide, on the corpus whose source reports explicit justification in $\sim 72\%$ of exploit attempts. An adversarial or merely tacit hacker can empty the channel at will; the companion’s reasoning-faithfulness caveat applies in full, and Section 6 differentiates it. The rung’s price is also visible: the detector is no longer content-free (13 English regexes that must be maintained per deployment language and scaffold), and the sidecar stores verbatim agent text.

E1 transfer to TRACE (2026-09-19). Both detectors, frozen exactly as developed on Terminal Wrench’s dev split, applied to the second reward-hacking corpus — different scaffold (a Claude-Code-shaped toolset), different task population, different hack provenance; the whole corpus is held-out because nothing was developed on it. Both collapse to base rate: **L1b 1.9% detection at 0.4% FPR, L3 1.5% at 1.2% FPR**. The per-pattern table is the instructive part: the justification vocabulary fires at equal rates in both classes (“hardcode”: 4.1% of hacked runs, 6.4% of *clean* ones), and verifier reconnaissance is virtually absent (1.9%). TRACE’s hacks — fallbacks, mocked results, expectations hardcoded inside ordinary edits — leave neither recon commands nor confession language. This is Section 6 measured rather than argued: Terminal Wrench’s 36–73% (L1b) and ~ 98 –100% (L3) quantify the detection of an *instructed, openly narrating* hacker, and neither number survives a corpus whose hackers had no reason to volunteer either signal. What does transfer is the protocol, not the patterns: dev-split development, frozen rules, held-out measurement — re-run per deployment. The recurring cost line of this paper is exactly that: content rungs buy detection only together with deployment-specific, maintained detector content.

Reward hacking: the detector type, not the rung, buys detection

Terminal Wrench, held-out detection per model; the frozen detectors re-applied to TRACE (different hack provenance) collapse to base rate

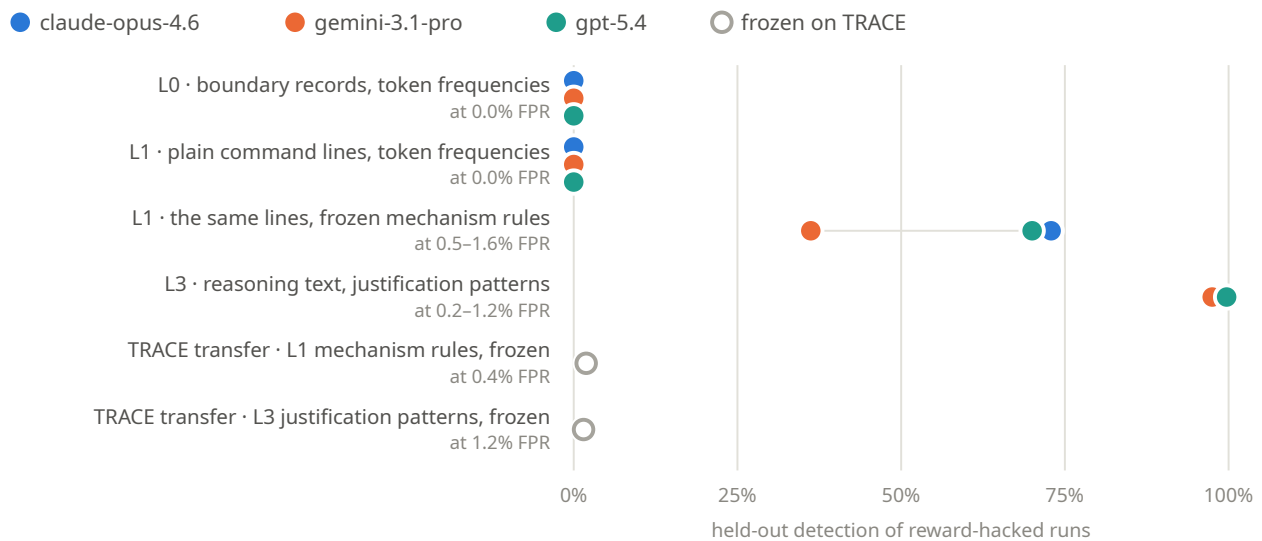


Figure 1. Reward hacking on Terminal Wrench: held-out detection per model for each rung–detector combination, with the same detectors frozen and re-applied to TRACE (hollow). Token frequencies detect nothing at L0 or L1; frozen mechanism rules over the *same* L1 lines reach 36–73% and justification patterns over L3 reasoning ~98–100% — and both collapse to base rate on TRACE, whose hackers neither probe the verifier nor narrate. The detector type, not the rung, buys the detection; the rules do not travel.

E2 (2026-09-20). The rung where the *cause* crosses the boundary. The companion’s Section 2.3 observed that an AgentDojo injection arrives as content inside a tool result; LO, which hashes that content away, measured 0–8% at 2–11% FPR. Six frozen content features over the inbound tool-result sidecars (`l2_text`) — the injection-template wrapper tag, second-person address from inside data, task-gating phrases, override wording, do-this-first wording, an imperative TODO; developed on the ~20% dev split, measured held-out on all 29 pipelines: **98.7% detection over the Lo-comparable successful-injection runs, 86.2% over all attacked runs, at 4.5% FPR**, dev and eval agreeing. The gap between the two denominators is structural, not a detector miss: only 73.1% of attacked runs ever get their injected string into a tool result (the agent never fetched the poisoned item), and a run the injection never reached is undetectable at this rung *by construction* — among reached runs, detection is 95.7%, and 100.0% in 24 of the 29 pipelines. The `transformers_pi_detector` pipeline makes the accounting visible: its defense suppresses delivery down to 118 of 484 attacked runs (26.2% detection over all attacked), and every one of the 118 that got through is flagged. Detection here is of the *attempted* injection at the boundary, whether or not the agent complied — which is why it beats the outcome-conditioned LO number by an order of magnitude: the cause is loud, template-shaped English sitting in stored content; only the consequence was quiet at LO.

The price tags mirror E1b’s, sharpened. The features are template-shaped: AgentDojo’s attacks come from a small set of English templates, and the rules encode that vocabulary — the `ignore_previous` rule survives only because it matches *before* the corpus’s own typo (“iunstructions”), a reminder of how brittle template matching is one character off its home corpus; after the TRACE transfer, the held-out number should be read as within-template-family generalization, not as a transferable detector. The two features made of generic English (“please do the following”, TODO) are the *only* false-positive sources — all 4.5% FPR comes from injection-shaped language occurring in legitimate tool output, the L2-specific

cost: at this rung the detector reads data someone else may have written, so the base-rate question (“how often does benign content look like an instruction?”) replaces calibration as the maintenance burden. And the rung’s storage price is the largest yet: the sidecars hold verbatim tool outputs — 137k lines including every injected payload — squarely the sensitive-content regime the cost section prices.

Injection: the rung that stores the cause detects it

AgentDojo pipelines, successful-injection runs — the same corpus and axes as the companion’s figure 3, one rung apart

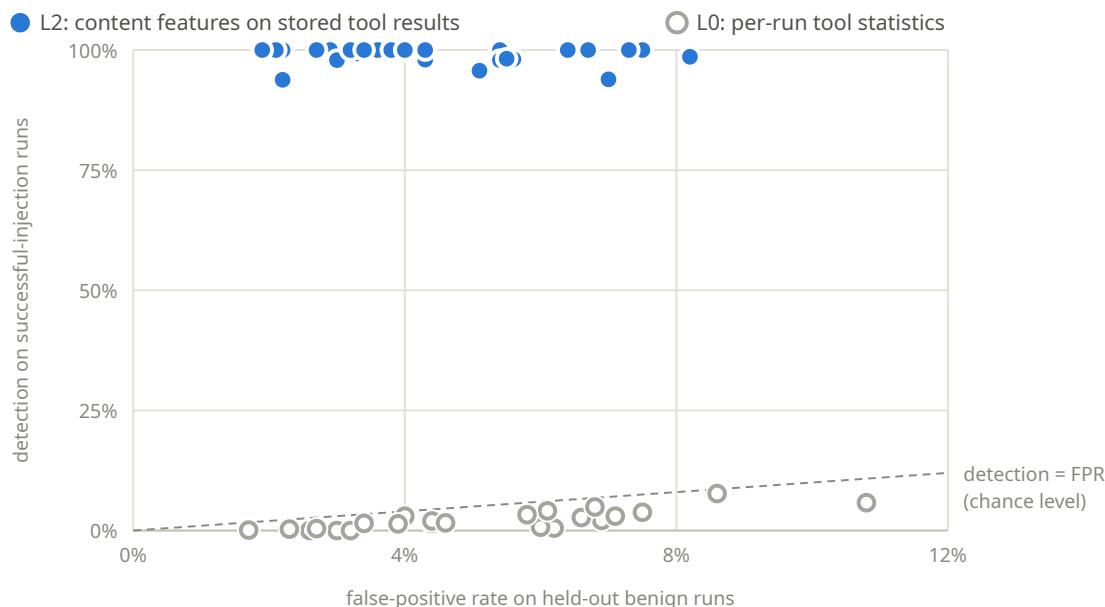


Figure 2. Injection detection versus false-positive rate for AgentDojo pipelines, on the same corpus and axes as the companion’s figure 3. Hollow gray: L0 per-run tool statistics, sitting on the chance diagonal (detection = FPR). Blue: six frozen L2 content features over inbound tool results, on the same successful-injection denominator. One rung — the one that stores the cause — moves every pipeline from chance to 94–100%.

E3 (2026-09-20). The corpus where the ladder’s cap becomes visible. TRAIL’s LO result was that the runtime status carries ~6% of the human-annotated error mass; E3 measures what the content rungs recover of the rest, with six frozen error-signature families (dev-split protocol as everywhere) and one deterministic attribution rule. Three findings.

The content rungs at the error’s location buy nothing. Signatures over the annotated span’s own stored outputs sit exactly on the noise floor — GAIA 6.0% coverage against 5.9% on non-annotated spans, SWE Bench 16.9% against 13.3% — and a 3-span downstream window does not help (GAIA 12.8% vs 17.0% noise). On SWE Bench nothing beats noise on any channel, for a structural reason worth stating: exception names and “not found” strings are ordinary *working vocabulary* in code-repair output, so error-shaped content tracks the base rate, not the errors.

The channel that does carry evidence is the prompt replay — an L1 channel, and only on GAIA. The smolagents scaffold routes tool observations into the next prompt: execution-error strings live almost exclusively in LLM `ll_text` (696 spans corpus-wide) and barely in any span’s stored outputs. Naive matching saturates there (history accumulates), but a delta rule — credit a location iff the *next* prompt contains more execution-signature matches than the last one before it — recovers **31.6% of GAIA’s annotated error mass against a 13.4% noise floor** (formatting errors 66%, tool-related 49%, context-handling 47%), held-out, dev and eval agreeing in shape. The rung lesson is scaffold-shaped: *where the observation flows decides which rung sees it*. On a scaffold that stores tool results as span

outputs this would have been an L2 result; here L2 is empty because the scaffold spends the observation on the prompt.

True context length corrects the Lo proxy. The spans record real prompt-token counts — the quantity the companion’s context-decay detector could only proxy by step index. Measured against it, the annotated error density per model call **falls** as context grows (GAIA: 0.33/call at 2–4k tokens to 0.10 at 16–32k; SWE Bench: 0.43 at 4–8k to 0.28 at 32–64k). This does not contradict the companion’s rising curve — that curve is the *status* signal, runtime tool failures over all spans, and it does rise. The two error masses diverge: machine-visible failures accumulate late, human-judged reasoning errors front-load (0.27/call in GAIA’s first ten steps). Only content access shows the divergence, and it reframes “context decay” on this corpus: the annotated mass is not a long-context degradation phenomenon; part of the front-loading may also be annotator localization behavior, which we cannot separate here.

The honest summary of E3 is therefore mostly a bound: after the execution-shaped slice (a quarter of GAIA’s mass at 2.4x the noise floor), the remaining annotated categories — instruction non-compliance, goal deviation, task orchestration, language-only — are *semantic judgments*, invisible to deterministic detectors at every rung. Seeing them would take an LLM judge, which is exactly the step both papers’ determinism rule refuses to take; 148 runs also keep all of this coverage accounting, not detection rates.

Most of the annotated error mass is invisible at every rung

TRAIL, held-out: share of human-annotated error locations a frozen content predicate covers, against the same predicate on non-annotated spans — only the scaffold-routed prompt-replay channel separates

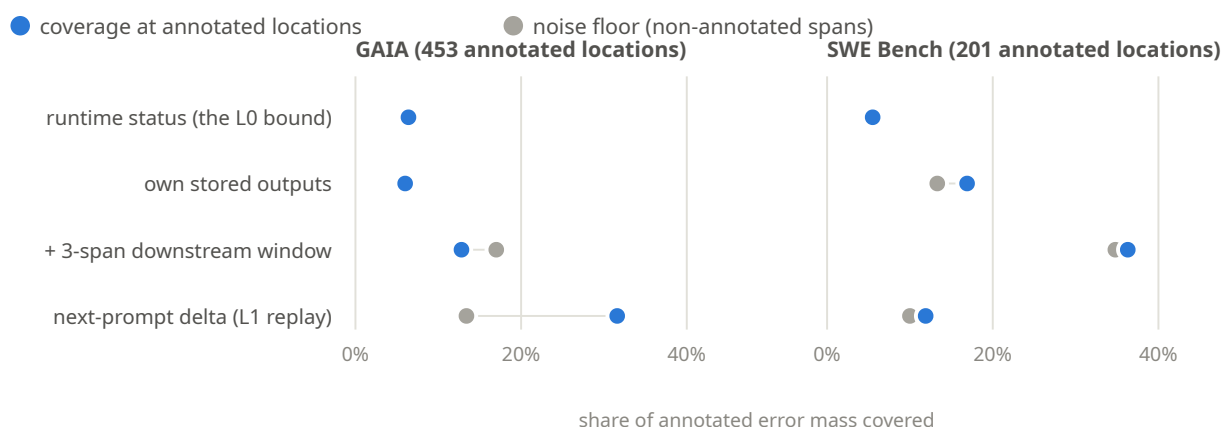


Figure 3. TRAIL, held-out: the share of the human-annotated error mass each channel covers (blue) against the same predicate’s rate on non-annotated spans (gray, the noise floor). Content at the error’s own location sits on its noise floor in both splits; only GAIA’s scaffold-routed next-prompt delta separates. The rest of the annotated mass — the semantic judgments — is invisible at every rung.

E4 (2026-09-21). The multi-agent rung, and a different *task*: the Who&When corpus holds only failures with a human attribution (mistake agent, mistake step), so the Lo measurement could establish only observability — the vocabulary exists at the seam. E4 asks what seam *content* (the message text, L3) buys for deterministic LOCALIZATION: one frozen prediction per run, measured as step@1 / agent@1 against the annotation, with content-free positional and structural predictors as the bar. Attribution accounting, never detection.

On the AG2-style expert teams (Algorithm-Generated, 110 held-out runs), content buys a real factor: the *causer heuristic* — name the record **before** the first visibly failing content (traceback, non-zero exitcode) — localizes the annotated mistake step in **30.9%** of runs against 14.5% for the best positional

baseline and 12.0% for a uniform pick, with agent@1 at 55.5%; dev and eval agree in shape. The mechanism is the corpus's loop: an expert writes code, the terminal reports the failure one step later, and the annotators blame the author — a causal step deterministic content can walk back.

On the Magentic-One runs (Hand-Crafted, 44 held-out runs), every content predictor sits at or below the positional baselines (before_first_error: 0.0%), and the reason is E3's semantic wall in attribution form: the annotated mistakes are judgments ("clicked an irrelevant link") that leave no error-shaped string, while the orchestrator's "Updated Ledger" cadence fires after every turn and carries no localization. What lift exists is *structural*: the first plain worker turn (a sequence property, no content read) reaches step@1 18.2% against 4.8% random and agent@1 59.1%. Content is not where this scaffold's attribution signal lives.

The pair is the rung lesson in miniature: the same detector class on the same corpus family splits by *failure mechanism* — walk-backable execution causality on one scaffold, deterministically invisible semantic judgment on the other. The remaining caveats: localization given failure is not detection (this corpus cannot price a detector's false-positive side at all); the dev split is 16 + 14 runs, so the held-out/dev agreement carries the weight; and true detection rates need an attribution corpus with a clean side — which is exactly what E4b adds.

E4b (2026-09-21). AgentHallu closes the E4 story with the piece Who&When could not provide: a clean side (443 hallucinated / 250 clean trajectories, 7 frameworks; used for paper measurements only — the repository's LICENSE says CC BY 4.0 while its project page says CC BY-NC-SA, and our ledgers are derivatives that stay unpublished either way). For the first time the attribution row gets real detection rates, and they are the paper's most instructive negative. The best dev-split rule — a *cross-channel consistency check*, tool -response error unacknowledged in the final message, a third detector type after distributions and content patterns — reached 73%/36% detection/FPR on BFCL's dev split and collapsed held-out to a weak, framework-inconsistent margin: **26.3% detection at 21.7% FPR aggregate, with a negative margin on three of seven frameworks.** Two structural reasons, both familiar. Four of the seven frameworks log *content only* — no tool responses, so the rule's channel does not exist and detection sits at zero, an observability floor set by the log format below every detector. And where the channel exists, the unacknowledged-error behavior turns out to be common *benign* behavior too: the semantic core of the label — is the final claim false? — is exactly what a deterministic rule cannot evaluate, the E3/E4 wall with its FP side finally priced. The localization transfer completes the symmetry: E4's causer heuristic, applied verbatim, degenerates to the first baseline (no execution loop to walk back — the error signature almost never fires), while what does localize is corpus-shaped *position* (Camel's annotations concentrate on its fixed third step: 58%). Who&When's 30.9% was the special case of executional failure, and AgentHallu measures how special: against hallucination — drift that fabricates content rather than crashing into it — the deterministic toolbox holds no detector, now shown at matched FPR rather than argued.

E5 (2026-09-21). The last experiment prices neither a rung nor a detector but the *realism* every earlier number was bought without. SWE-chat records 5,851 real working sessions of 190 identified users — real wall-clock timestamps, real CLI updates mid-timeline, real task churn — and its Claude Code slice (4,771 sessions, 327k boundary records) supports the companion's protocol unchanged: one deployment = one user, sessions as runs, 50/25/25 splits, thresholds calibrated out-of-sample. Three measurements, one per caveat the composed-stream experiments had to disclose.

The composed-timeline caveat itself prices at approximately zero. On 65 users' longest same-CLI-version blocks, the chronological arm and the session-shuffled arm produce the same false-alarm rates within a few points (divergence 6% vs 5%, CUSUM 34% vs 30% at margin 2.0; 34/32 and 48/45 at 1.2). Every earlier stream experiment's honesty note — "stream order is seeded shuffling" — turns out to have cost nothing where it could be measured: within a stationary regime, exchangeable composition neither hides nor manufactures false alarms. What is expensive is the regime itself: a third of CUSUM channels alarm on real single-user, single-version timelines at margin 2.0, where the same protocol on SWE-bench's benchmark batches sat mostly at 0–30% over horizons an order of magnitude longer. Real deployments are noisier in-control than any benchmark corpus in this project.

Real version drift disappears into that noise. The corpus's 19 verified in-deployment CLI updates (empty and unattributed version strings excluded) are the first version-drift measurement on records whose boundary nobody constructed — and detection at those boundaries (26% / 47% divergence/CUSUM at margin 2.0) is statistically indistinguishable from the no-change control at matched geometry (17% / 55%). The reason is in the effect sizes: the median across-boundary JSD is 0.132, the median *within-version* churn at a fake boundary 0.127 — a CLI patch moves a user's tool distribution no further than their own ordinary fortnight. This does not contradict the companion's 80–100%: SWE-bench's boundaries were scaffold-level submission changes measured batch-against-batch with the task mix held fixed, and its $\text{JSD} \geq 0.02$ detection line is real *at that scale*. The two results bracket the phenomenon instead: agent-version changes detect when they move behavior above the deployment's in-control churn, and on real per-user timelines that churn sits near $\text{JSD} \sim 0.13$ — six times the companion's detection line — so patch-level updates are behaviorally sub-noise, the OpenHands model-swap result reproduced in the wild.

The per-deployment premise survives; the frozen baseline ages. The fingerprint accounting is the one unqualified positive: within-user JSD between adjacent 400-record blocks (median 0.149) sits well below between-user JSD at identical sample size (0.358, 1,653 pairs) — users are real, separable behavioral identities at the boundary, which is the premise every per-deployment baseline in both papers rests on. But the same table prices baseline lifetime: against a user's first block, JSD rises from 0.137 at a ~2-day lag toward ~0.2 within about two weeks — drift is the steady state, and an admission-time baseline on a real deployment is a perishable object. The honest reading of E5 is symmetric: the Lo machinery's *methodology* transfers to reality better than we could previously claim (composition costs nothing, fingerprints exist), while its headline *detection* result inherits a noise floor an order of magnitude above the benchmark one. Caveats: the analysis covers one scaffold family's active users (min-block filters select for volume), 34% of sessions are excluded as unattributable, and task mix is deliberately uncontrolled — that confound is not a flaw in the experiment but the definition of deployment reality, and no monitor at the boundary gets to remove it.

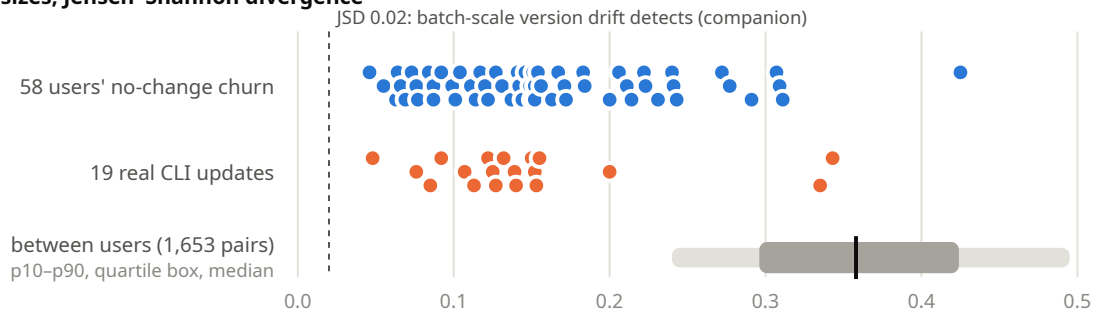
Real timelines: in-control churn dwarfs the version-drift line

SWE-chat Claude Code users, 400-record blocks. Real CLI updates (orange) sit inside the same users' no-change churn; only user identity separates

● one user, adjacent blocks (churn)

● real CLI-version boundary

(a) effect sizes, Jensen-Shannon divergence



(b) the frozen baseline ages: JSD of block k vs a user's first block

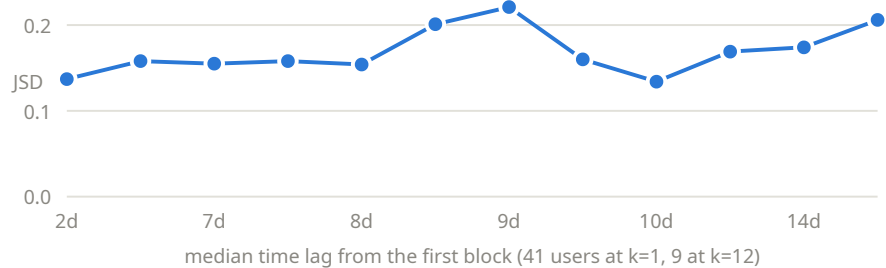


Figure 4. SWE-chat real timelines, 400-record blocks. (a) Effect sizes: real CLI-version boundaries (orange) sit inside the same users' no-change churn (blue), and both sit an order of magnitude above the companion's batch-scale detection line (dashed); only user identity — the between-user band — separates. (b) Against a user's first block, within-user JSD rises from 0.137 toward ~0.2 within about two weeks: an admission-time baseline on a real deployment is a perishable object.

5. The dividing line

Read together, the five families sort every measured number along a single axis, and it is not the rung axis. What decides detection is whether the drift leaves an **executorial trace in stored bytes**, and whether the rules that read it live on their home corpus:

WHERE THE EVIDENCE LIVES	MEASURED (HELD-OUT)	VERDICT
mechanism traces, home corpus (E1b: recon commands, tampering)	36–73% at 0.5–1.6% FPR	detects — with authored, per-deployment rules
volunteered narration, home corpus (E1/L3)	97.5–99.7% at $\leq 1.2\%$ FPR	detects — a ceiling for non-hiding agents
the cause itself stored as content (E2: injection)	98.7% at 4.5% FPR	detects — within its template family
executorial causality at the seam (E4, Algorithm-Generated)	step@1 30.9% vs. 12.0% random	localizes
the same rules, one corpus over (E1 → TRACE)	1.5–1.9%	collapses to base rate
semantic judgment (E3 beyond the executorial slice; E4 Hand-Crafted)	at the noise floor	invisible at every rung

WHERE THE EVIDENCE LIVES	MEASURED (HELD-OUT)	VERDICT
hallucination, clean side measured (E4b)	26.3% at 21.7% FPR	no deterministic separation
real in-control churn (E5)	boundary JSD 0.132 vs. churn 0.127	version drift sub-noise

The top half is one phenomenon: **executorial failure detects**. Every positive number in this paper is a rule reading a physical residue of the failure mechanism — a recon command, a confession, an injected template, a traceback one step after its author — and every one of them was bought with deployment-specific detector content whose off-corpus value the TRACE transfer measured at base rate. The bottom half is the other phenomenon: **judgment failure is invisible**. Where the label is a semantic evaluation — was the claim false, was the link irrelevant, did the plan deviate — no deterministic predicate at any rung separates it from benign behavior at matched FPR, and E4b finally prices the false-positive side of pretending otherwise. E5 adds an orthogonal ceiling that binds even the executorial side on real deployments: an effect must clear in-control churn (JSD ≈ 0.13 on real per-user timelines) before any detector, at any rung, can see it.

The ladder’s honest summary is therefore not “content buys detection” but: *content buys detection of executorial drift, per deployment, for the lifetime of its rules — and buys nothing against semantic drift under the determinism rule both papers keep*. Whether that boundary is acceptable is a deployment decision; the point of this paper is that it is now a measured boundary, not an argued one.

6. What reasoning access can and cannot promise

The E1/L3 number is reported as a ceiling, and the caveat deserves to be differentiated rather than footnoted, because the rungs differ not only in cost but in *adversarial robustness*.

Written reasoning is working memory, not a log. A transformer performs substantial computation inside each forward pass that never surfaces as text; a simple exploit (“the checker only reads the file, so I write the file”) needs no written plan at all. What the reasoning channel records is not what the model thought but what it chose to use as scratchpad. This cuts both ways, and the second direction is the strongest honest claim L3 supports: per-step silent capacity is bounded, multi-step exploits degrade without a scratchpad, so a monitored channel imposes a real cost — the agent must either write something or hack worse.

The channel empties by omission long before it needs steganography. The cheapest evasion is not obfuscated reasoning but neutral phrasing: “I create a script that sets the expected values” with the incriminating clause (“... instead of parsing the XML as required”) simply left off. Our 13 patterns are powerless against exactly this, and it costs the agent nothing. The chain-of-thought-faithfulness literature adds that written justifications need not reflect the deciding computation even *without* adversarial intent; encoded or steganographic reasoning is the exotic endpoint, needed only when a task is too complex for silent computation *and* the text is monitored.

The rung observes a channel, not a mind. `l3_text` is what the scaffold logs (the ATIF message field). Provider-side reasoning tokens that never reach the scaffold are invisible to this rung by construction — L3 measures what the agent volunteers into the logged channel, nothing deeper.

The contrast with Lo is the point. Boundary records cannot be emptied by the agent: what it *does* is observed at the boundary; what it *thinks* is volunteered. Terminal Wrench’s hackers are goal-

conditioned to exploit and narrate it openly, which is why the L3 ceiling sits at ~98–100% — every one of those runs would keep its Lo/L1 behavioral trace unchanged if the agent stopped narrating. A deployment story that leans on L3 alone therefore inherits the agent’s cooperation as an assumption; the ladder’s honest reading is L3 as a cheap, high-yield tripwire for non-hiding agents on top of boundary records, never as their replacement.

7. Cost side

The ladder’s price was named qualitatively in Section 1; the experiments let most of it be *measured*. Five cost classes, in the order a deployment meets them.

Storage. Measured on our own ledgers (uncompressed JSONL, apparent size):

CORPUS	L0 LEDGER	CONTENT SIDECAR	MULTIPLIER
Terminal Wrench	33 MB	114 MB (L1+L3)	3.5×
AgentDojo	63 MB	211 MB (L2)	3.3×
TRAIL	1.8 MB	81 MB (L1–L3 spans)	45×
TRACE	2.5 MB	11 MB (L1)	4.4×
Who&When	1.3 MB	5.6 MB (L3)	4.3×

The multiplier is 3–4× when the sidecar holds one channel and ~45× when it holds everything the scaffold logged (TRAIL’s spans store prompts and outputs). The reference point below Lo matters more: SWE-chat’s raw transcripts are 9.7 GB; the Lo ledger distilled from them is 96 MB — about 1%. Boundary records are not a cheap *version* of the log; they are a different storage class.

Sensitivity and redaction. An Lo ledger stores names, hashes, status codes and timestamps — no payload a redaction policy could apply to. Each content rung stores what someone wrote: the AgentDojo sidecars contain every injected payload verbatim, the Terminal Wrench and TRACE sidecars contain repository contents, and TRAIL’s spans contain full prompts. E5 is the demonstration that the distinction is operational, not rhetorical: SWE-chat is the one corpus of *real user work*, and its entire evaluation ran with parameters reduced to sha256 + length at export time — the only corpus where privacy was at stake is the one where Lo sufficed. Redaction of content sidecars, where required, is not free either: it rewrites the bytes a fingerprint names, so a redacted sidecar is a *new* evidentiary object, severed from verdicts computed on the original.

Detector content. The distributional channels need calibration but no authored content: the same code ran unchanged on every corpus. Every content-rung detection in this paper instead required deployment-specific detector content — 13 justification regexes (L3), 5 mechanism rules (L1b), 6 injection features (L2) — and the TRACE transfer measured what that content is worth off its home corpus: 1.5– 1.9%, base rate. The recurring cost of the content rungs is therefore not the rules’ first authoring but their *re*-authoring per deployment and their maintenance as the scaffold, task mix and language move. At L2 a second recurring line appears: the detector reads data outsiders write, so the benign base rate of instruction-shaped content (all of E2’s 4.5% FPR) has to be monitored per deployment, where the Lo channels only ever needed a calibration stream.

Compute. Widening the observation explodes the vocabulary (~20 Lo tools → 51k–109k L1 tokens on Terminal Wrench), and the naive stream evaluation that is instant at Lo became infeasible at L1 (a

single model’s pass did not finish in 10 hours) until the incremental variants were written (3m37s for the same table; Section 4, E1). The window machinery also stops transferring: a 50-token window saturates over an L1 vocabulary, and de-saturating it costs a 64× longer window (E1 sweep) — rung changes silently invalidate stream hyperparameters that looked settled at Lo.

Reproducibility and shareability. Determinism survives every rung — frozen rules over stored bytes replay bit-identically, which is the property both papers refuse to spend. What the rungs *do* erode is shareability: an Lo ledger carries no content, so only its source’s license constrains it (AgentHallu’s contradictory licensing keeps even its Lo ledgers unpublished here); a content sidecar carries the source’s *text*, and an operational deployment’s sidecars would be confidential by default. A replication story that needs the content rungs is therefore a story about *access*, not just about disk.

References

Corpora (entries shared with the companion are cited identically there):

- Baumann, Padmakumar, Li, Yang, Yang, Koyejo. SWE-chat: real-world AI coding sessions in the wild. arXiv:2604.20779, 2026. Dataset ODC-BY.
- Bercovich, Segal, Zhang, Saxena, Raghunathan, Zhong. Terminal Wrench: a dataset of 331 reward-hackable environments and 3,632 exploit trajectories. arXiv:2604.17596, 2026.
- Debenedetti, Zhang, Balunović, Beurer-Kellner, Fischer, Tramèr. AgentDojo: a dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. NeurIPS 2024 Datasets and Benchmarks; arXiv:2406.13352.
- Deshpande, Gangal, Mehta, Krishnan, Kannappan, Qian. TRAIL: trace reasoning and agentic issue localization. arXiv:2505.08638, 2025.
- Deshpande, Kannappan, Qian. Benchmarking reward hack detection in code environments via contrastive analysis (the TRACE dataset). arXiv:2601.20103, 2026.
- Liu, Yang, Li, Li, He. AgentHallu: benchmarking automated hallucination attribution of LLM-based agents. arXiv:2601.06818, 2026. (Repository LICENSE CC BY 4.0; project page CC BY-NC-SA 4.0 — used here for paper measurements only, no data redistributed.)
- Zhang, Yin, Zhang, Liu, Han, Zhang, Li, Wang, Wang, Chen, Wu. Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems (the Who&When dataset). arXiv:2505.00212, 2025.

Methods and context:

- Eckel, Radehaus. Behavioral drift in autonomous LLM-driven systems: a survey of detection approaches and the case for deterministic detection (the companion survey; its Section 7 provides every Lo baseline used here). 2026.
- Jimenez, Yang, Wettig, Yao, Pei, Press, Narasimhan. SWE-bench: can language models resolve real-world GitHub issues? ICLR 2024; arXiv:2310.06770. (Source of the companion’s version-drift baselines referenced in Section 1.)